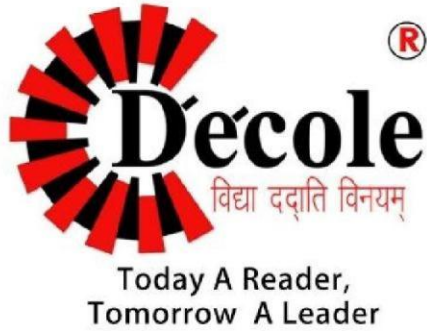




Dezyne École College
www.dezyneecole.com

Title of Project

Assignment File

Subject Name

Data Structure and C++

Submitted By

Meenakshi Mali

Department - BCA

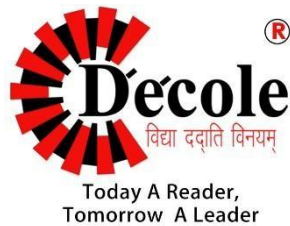
Semester – 2nd

Session – 2025-26

Project Report of
Data Structure and C++

Submitted to
Dezyne École College
Towards the Partial Fulfilment of
BCA 2nd Semester

By
Meenakshi Mali



www.dezyneecole.com

2025-2026

Acknowledgement

We take this opportunity to express our sincere gratitude to Dezyne École College for providing us with the knowledge, resources, and guidance essential to the successful completion of this project.

Our deepest thanks to our respected Principal, Dr. Vinita Mathur, for her visionary leadership and continuous encouragement. Her commitment to academic excellence and student growth has been a constant source of motivation and pride.

We are equally thankful to all our faculty members for their consistent support, expert advice, and valuable insights that shaped our learning and helped us achieve this milestone.

Lastly, we extend heartfelt thanks to everyone who directly or indirectly contributed to the success of this project.

We are proud to be part of Dezyne École College, an institution that truly nurtures professionalism, innovation, and purpose.

With sincere appreciation,

Meenakshi Mali

Department of BCA

Dezyne École College, Ajmer

1. WAP to input size of Queue from user and create a queue using array. And perform push, pop and traverse operations using menu driven programming and functions.

Code:

```
#include<iostream>
```

```
#include<conio.h>
```

```
using namespace std;
```

```
class Queue
```

```
{
```

```
public:
```

```
    int QUEUE[10];
```

```
};
```

```
class QueueOperation : public Queue
```

```
{
```

```
public:
```

```
    int Front, Rear, MaxSize;
```

```
    QueueOperation()
```

```
{
```

```
        Front = Rear = -1;
```

```
        cout << "Enter Size of Queue (1-10) -- ";
```

```
        cin >> MaxSize;
```

```

if(MaxSize > 10 || MaxSize <= 0)
{
    cout << "Invalid Queue Size...";
    exit(0);
}
}

void Push()
{
    system("cls");

    cout << "Push Operation Module\n\n";

    int DATA;

    // Overflow Condition
    if(Rear == MaxSize - 1)
    {
        cout << "Queue Overflow...\n\n";
        return;
    }

    cout << "Enter Data -- ";
    cin >> DATA;

    // First Element

```

```

    if(Front == -1)
    {
        Front = Rear = 0;
    }
    else
    {
        Rear++;
    }

    QUEUE[Rear] = DATA;

    cout << "\nData Inserted Successfully...\n\n";
}

void Pop()
{
    system("cls");

    cout << "Pop Operation Module\n\n";

    // Underflow Condition
    if(Front == -1)
    {
        cout << "Queue Underflow...\n\n";
        return;
    }

```

```
cout << "Deleted Element -- " << QUEUE[Front] << endl;

// Only One Element
if(Front == Rear)
{
    Front = Rear = -1;
}
else
{
    Front++;
}

cout << "\nData Deleted Successfully...\n\n";
}

void Peek()
{
    system("cls");

    cout << "Display Queue Module\n\n";

    if(Front == -1)
    {
        cout << "Queue is Empty...\n\n";
        return;
    }
}
```

```
}
```

```
cout << "Queue Elements are -- \n\n";
```

```
for(int i = Front; i <= Rear; i++)
```

```
{
```

```
    cout << QUEUE[i] << " -> ";
```

```
}
```

```
cout << "NULL\n\n";
```

```
}
```

```
void MainMenu()
```

```
{
```

```
    while(true)
```

```
    {
```

```
        system("cls");
```

```
        cout << "\tQUEUE OPERATIONS\n\n";
```

```
        cout << "1. Push\n";
```

```
        cout << "2. Pop\n";
```

```
        cout << "3. Display Queue\n";
```

```
        cout << "4. Exit\n\n";
```

```
        cout << "Choose Any Option -- ";
```



```
char c = getch();
```

```
switch(c)
```

```
{
```

```
    case '1':
```

```
        Push();
```

```
        break;
```

```
    case '2':
```

```
        Pop();
```

```
        break;
```

```
    case '3':
```

```
        Peek();
```

```
        break;
```

```
    case '4':
```

```
        exit(0);
```

```
    default:
```

```
        cout << "\nInvalid Choice...\n";
```

```
}
```

```
cout << "\nPress Any Key To Continue...";
```

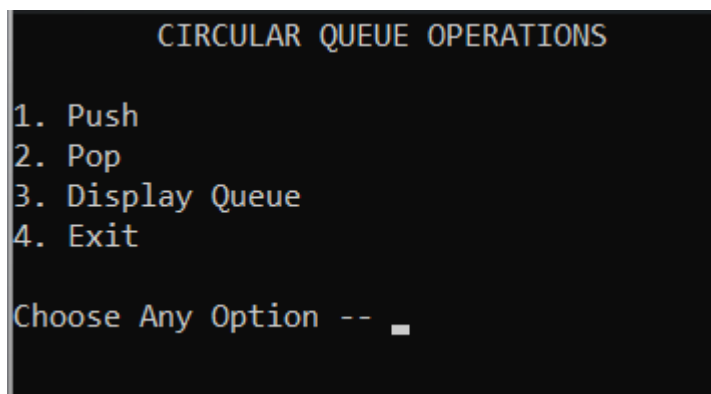
```
getch();
```

```
    }  
}  
};
```

```
int main()  
{  
    QueueOperation Q;  
    Q.MainMenu();  
  
    return 0;  
}
```

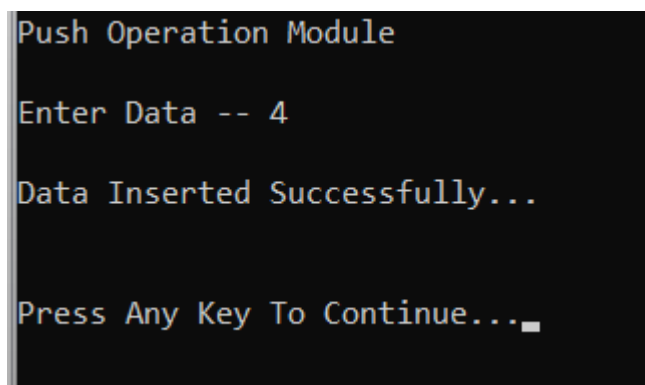
Output:

Menu



```
          CIRCULAR QUEUE OPERATIONS  
  
1. Push  
2. Pop  
3. Display Queue  
4. Exit  
  
Choose Any Option -- █
```

Push



```
Push Operation Module  
  
Enter Data -- 4  
  
Data Inserted Successfully...  
  
Press Any Key To Continue... █
```

Pop

```
Pop Operation Module

Deleted Element -- 4

Data Deleted Successfully...

Press Any Key To Continue...
```

Peek

```
Display Circular Queue Module

Queue Elements are --

4 -> 5 -> 8 -> NULL

Press Any Key To Continue...■
```

2.WAP to input size of Circular Queue from user and create a circular queue using array. And perform push, pop and traverse operations using menu driven programming and functions.

Code:

```
#include<iostream>

#include<conio.h>

using namespace std;

class CQueue
{
public:
    int CQUEUE[10];
};

class CQueueOperation : public CQueue
```

```

{
public:
    int Front,Rear,MaxSize;
    CQueueOperation()
    {
        Front = Rear = -1 ;
        cout<<"Enter Size of Queue -- ";
        cin>>MaxSize;
    }
    int Push()
    {
        system("cls");
        cout<<"Push Operation Module \n\n";
        int DATA , PushStatus=0;
        cout<<"Enter Data - ";
        cin>>DATA;
        if(Front == 0 && Rear == MaxSize-1){
            cout<<"Queue is Overflow...\n\n";
            return 0;
        }
        if(Front == -1 && Rear == -1){
            Front = Rear = 0;
            CQUEUE[Rear] = DATA;
            PushStatus = 1 ;
        }
        else if (Rear == MaxSize-1 && Front!= 0){

```

```

    Rear = 0;
    CQUEUE[Rear] = DATA;
    PushStatus = 1;
}
else {
    Rear = Rear + 1;
    CQUEUE[Rear] = DATA;
    PushStatus = 1;
}
if(PushStatus == 1)
    cout<<"\n\nData Successfully Inserted/Pushed..\n\n";
}

int Pop()
{
    system("cls");
    cout<<"Pop From Queue Module \n\n";
    if(Front == -1 && Rear == -1){
        cout<<"Queue is Underflow...\n\n";
        return 0;
    }
    if(Front == Rear){
        CQUEUE[Front] = 0;
        Front = Rear = -1;
    }else if( Front == MaxSize-1) {
        CQUEUE[Front] = 0;
        Front = 0;
    }
}

```

```

    }
else {
    CQUEUE[Front]= 0;
    Front = Front + 1;
}
cout<<"Data Deleted Successfully...\n\n";
}
int Peek()
{
    if(Front == -1){
        cout<<"Queue is Empty..\n\n";
        return 0;
    }
    cout<<"Queue Element are -- \n\n";
    for(int POS = Front ; 1 ;){
        cout<<POS<<" - "<<CQUEUE[POS]<<" -> ";
        if( POS == Rear){
            break;
        }
        POS = (POS = MaxSize-1 && POS != Rear)?0:++POS;
    }
    cout<<"\n\n";

}

void MainMenu()
{

```

```

system("cls");
cout<<"\tCircular Queue Operation -- \n\n";
cout<<"1.Push..\n";
cout<<"2.Pop..\n";
cout<<"3.Peek..\n";
cout<<"4.Exit..\n";
cout<<"Choose any one of these..\n\n";
char c = getch();
if(c == '1'){
    Push();
}else if (c == '2'){
    Pop();
}else if (c == '3'){
    Peek();
} else if (c == '4'){
    exit(0);
}
cout<<"Do you want to continue press 1 else any key...\n";
c = getch();
if(c == '1'){
    MainMenu();
}
}
};

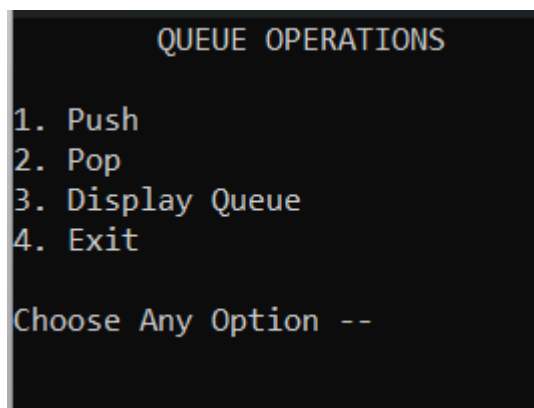
int main()
{

```

```
CQueueOperation C;  
C.MainMenu();  
}
```

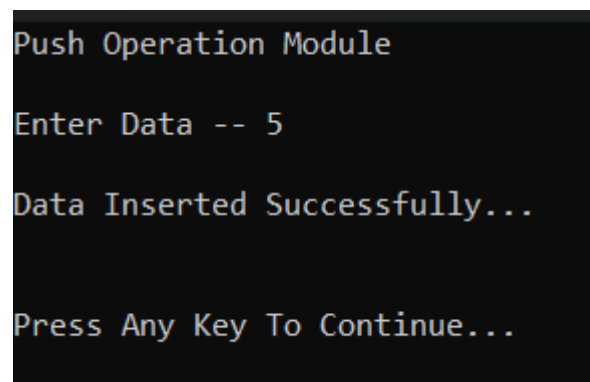
Output:

Menu



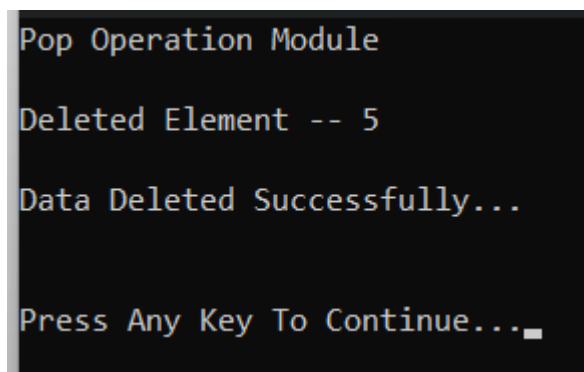
```
          QUEUE OPERATIONS  
  
1. Push  
2. Pop  
3. Display Queue  
4. Exit  
  
Choose Any Option --
```

Push



```
Push Operation Module  
  
Enter Data -- 5  
  
Data Inserted Successfully...  
  
Press Any Key To Continue...
```

Pop



```
Pop Operation Module  
  
Deleted Element -- 5  
  
Data Deleted Successfully...  
  
Press Any Key To Continue..._
```

Peek


```
Display Queue Module

Queue Elements are --

5 -> 8 -> 9 -> NULL

Press Any Key To Continue...
```

3.WAP to input size of Stack from user and create a stack using array. And perform push, pop and peek operations using menu driven programming and functions.

Code:

```
#include<iostream>
```

```
#include<conio.h>
```

```
using namespace std;
```

```
class Stack
```

```
{
```

```
public:
```

```
    int STACK[10];
```

```
};
```

```
class StackOperation : public Stack
```

```
{
```

```
public:
```

```
    int TOP, MaxSize;
```

```

StackOperation()
{
    TOP = -1;

    cout << "Enter Size of Stack (1-10) -- ";
    cin >> MaxSize;

    if(MaxSize > 10 || MaxSize <= 0)
    {
        cout << "Invalid Stack Size...";
        exit(0);
    }
}

void Push()
{
    system("cls");

    cout << "Push In Stack Module\n\n";

    // Overflow Condition
    if(TOP == MaxSize - 1)
    {
        cout << "Stack Overflow...\n\n";
        return;
    }
}

```

```
int DATA;
```

```
cout << "Enter Data -- ";
```

```
cin >> DATA;
```

```
TOP++;
```

```
STACK[TOP] = DATA;
```

```
cout << "\nData Pushed Successfully...\n\n";
```

```
}
```

```
void Pop()
```

```
{
```

```
system("cls");
```

```
cout << "Pop From Stack Module\n\n";
```

```
// Underflow Condition
```

```
if(TOP == -1)
```

```
{
```

```
    cout << "Stack Underflow...\n\n";
```

```
    return;
```

```
}
```

```
cout << "Deleted Element -- " << STACK[TOP] << endl;
```

```
STACK[TOP] = 0;
```

```
TOP--;
```

```
cout << "\nData Deleted Successfully...\n\n";
```

```
}
```

```
void Peek()
```

```
{
```

```
    system("cls");
```

```
    cout << "Display Stack Module\n\n";
```

```
    if(TOP == -1)
```

```
    {
```

```
        cout << "Stack is Empty...\n\n";
```

```
        return;
```

```
    }
```

```
    cout << "Stack Elements are -- \n\n";
```

```
    for(int i = TOP; i >= 0; i--)
```

```
    {
```

```
        cout << STACK[i] << " -> ";
```

```
}
```

```
cout << "NULL\n\n";
```

```
}
```

```
void MainMenu()
```

```
{
```

```
while(true)
```

```
{
```

```
system("cls");
```

```
cout << "\tSTACK OPERATIONS\n\n";
```

```
cout << "1. Push\n";
```

```
cout << "2. Pop\n";
```

```
cout << "3. Display Stack\n";
```

```
cout << "4. Exit\n\n";
```

```
cout << "Choose Any Option -- ";
```

```
char c = getch();
```

```
switch(c)
```

```
{
```

```
case '1':
```

```
    Push();
```

```
        break;

    case '2':
        Pop();
        break;

    case '3':
        Peek();
        break;

    case '4':
        exit(0);

    default:
        cout << "\nInvalid Choice...\n";
    }

    cout << "\nPress Any Key To Continue...";
    getch();
}

};

int main()
{
    StackOperation S;
```

```
S.MainMenu();

return 0;
}
```

Output:

Menu

```
STACK OPERATIONS
1. Push
2. Pop
3. Display Stack
4. Exit
Choose Any Option -- █
```

Push

```
Push In Stack Module
Enter Data -- 4
Data Pushed Successfully...
Press Any Key To Continue...
```

Pop

```
Pop From Stack Module
Deleted Element -- 8
Data Deleted Successfully...
Press Any Key To Continue... █
```

Peek

```
Display Stack Module

Stack Elements are --

8 -> 6 -> 4 -> NULL

Press Any Key To Continue..._
```

Q4. WAP that can perform Factorial , Fibonacci and Tower of Hanoi operation using object oriented programming.

Code:

```
#include<iostream>

#include<conio.h>

using namespace
std; class Ass19
{
public:
    int Fact(int n)
    {
        if(n == 1)
return 1;
else
        return n * Fact(n-1);
    }
    int Fibo(int n)
    {
        if(n == 0)
return 0;
```



```

else if(n == 1){
return 1;
    }
    else
        return Fibo(n-1)+Fibo(n-2);
}

void towerofhanoi(int n , char source , char auxiliary , char destination)
{
    if(n == 1){
        cout<<"Move disk 1 from "<<source<<" to "<<destination<<endl;
        return;
    }

    towerofhanoi(n-1,source , destination,auxiliary);    cout<<"Move
disk "<<n<<" from "<<source<<" to "<<destination<<endl;
    towerofhanoi(n-1,source,auxiliary,destination);

}

void MainMenu()
{
    system("cls");

    cout<<"Choose Any One Of The
Following:\n";    cout<<"1.Find Factorial\n";
    cout<<"2.Print Fibonacci\n";
    cout<<"3.Tower of Hanoi\n";
    cout<<"4.Exit\n\n";    char c = getch();

    if (c == '1'){
        cout<<"Find Factorial Module ";

        int n;

        cout<<"Enter a Number : ";

        cin>>n;
    }
}

```

```

        cout<<"Factorial of "<<n<<" is ";
cout<<Fact(n);
    }
    else if ( c == '2'){
        cout<<"Find Fibonacci Module ";
        int limit;
        cout<<"Enter a Limit : ";
cin>>limit;

        cout<<"Fibonacci Series -- \n";
        for(int i = 0 ; i < limit ; i++){
cout<<Fibo(i)<<" ";
        }
    }
    else if (c == '3'){
        cout<<"Tower of Hanoi Module..\n";
        cout<<"Enter limit(3-6) -- ";
        int n ;      cin>>n;
towerofhanoi(n,'A','B','C');
    }
    else if (c == '4'){
exit(0);
    }

    cout<<"\n\nDo you want to continue press 1 else any key ";
    c = getch();
    if(c == '1')
        MainMenu();
    }
};

int main()

```

```
{  
    Ass19 R;  
    R.MainMenu();  
}
```

Output: Menu

```
Choose Any One Of The Following:  
1.Find Factorial  
2.Print Fibonacci  
3.Tower of Hanoi  
4.Exit
```

Factorial

```
Choose Any One Of The Following:  
1.Find Factorial  
2.Print Fibonacci  
3.Tower of Hanoi  
4.Exit  
  
Find Factorial Module Enter a Number : 3  
Factorial of 3 is 6  
  
Do you want to continue press 1 else any key
```

Fibonacci

```
Choose Any One Of The Following:  
1.Find Factorial  
2.Print Fibonacci  
3.Tower of Hanoi  
4.Exit  
  
Find Fibonacci Module Enter a Limit : 8  
Fibonacci Series --  
0 1 1 2 3 5 8 13  
  
Do you want to continue press 1 else any key _
```

Tower of Hanoi

Choose Any One Of The Following:

- 1.Find Factorial
- 2.Print Fibonacci
- 3.Tower of Hanoi
- 4.Exit

Tower of Hanoi Module..

Enter limit(3-6) -- 3

Move disk 1 from A to C

Move disk 2 from A to B

Move disk 1 from A to B

Move disk 3 from A to C

Move disk 1 from A to B

Move disk 2 from A to C

Move disk 1 from A to C

Do you want to continue press 1 else any key



Thank You

Meenakshi Mali
BCA 2nd Semester
Dezyne École College
